

INFORME PROYECTO



MATRICULATOR

Reconocimiento y validación de matrículas

Equipo conformado por:

Organización y Gestión	Carmelo José Beato Garcia
Programador	Oihan Cabada

Fecha de inicio: 21/09/2026

Última actualización: 05/10/2026

Ubicación: En centro de formación profesional en CEBANC, San Sebastián.

En este informe se ha utilizado inteligencia artificial.

Índice

Índice.....	2
Aplicación elegida.....	3
Matriculator, detector de matrículas.....	3
Comparación de lenguajes y toma de decisión.....	4
Ventajas e inconvenientes de cada lenguaje.....	4
Programación.....	6
Necesidades de la IA en nuestra aplicación:.....	6
1. Inicio y captura de la imagen.....	7
2. Validación inicial del archivo.....	7
3. Detección de la matrícula.....	7
4. Extracción y validación del texto (OCR).....	7
5. Verificación de confianza y revisión.....	7
6. Finalización y salida.....	7
Etapas de antes de utilización de la IA y después.....	8
1. Antes de integrar el modelo de IA.....	8
Componentes principales.....	8
2. Después de entrenar o integrar el modelo.....	9
Diagrama de re entreno.....	9
Pseudocódigo empleado.....	10
Lenguajes de marcado y formatos de datos.....	11
Preguntas a contestar.....	12

Aplicación elegida

Matriculator, detector de matrículas

Matriculator es una aplicación que detecta matrículas en imágenes de prueba. El funcionamiento es el siguiente: una cámara con visión artificial es capaz de detectar las matrículas de los coches. La cámara trabaja de manera trivial con la luz ambiental. Sin embargo, cuando es de noche, o se oscurece de manera repentina, la cámara necesita apoyo de una luz artificial para poder detectar las matrículas. Una vez detectada la matrícula, la cámara toma automáticamente una foto de la misma y la manda al sistema de IA informático para ver si es una matrícula válida o no. En cada caso, el sistema le indica al usuario si es o no válida la matrícula. En el caso de ser válida, la aplicación extrae la siguiente información: la fecha de la captura, el código de la matrícula, el tipo de placa, si pertenece a España y si pertenece a otro país.

La idea es que estos equipos esten puestos en las entradas y salidas de los parkings, aunque estos pudiendo ser abiertos o cerrados. Con ello se podrá llevar un control exacto de los coches que entran y salen del parking, de tal manera que se sabe cuando el parking está lleno, si hay algun incidente, hay más información con la que trabajar y por supuesto, se podrá hacer un seguimiento para el cobro por el servicio. Esto es interesante para establecimientos que requieran una seguridad extra, como lo pueden ser centros comerciales, aeropuertos, hospitales o estadios, entre otros. Además, en el futuro podemos implementar mejoras adaptativas para que se puedan usar radares de tráfico para mejorar la seguridad vial. Incluso se puede considerar poner esta tecnología a disposición de radares estrategicos cerca de las fronteras, para monitorear quien entra y quien sale del país. Para la seguridad nacional es un avance importante.

Comparación de lenguajes y toma de decisión

Ventajas e inconvenientes de cada lenguaje

- **Python:** Tiene muchas bibliotecas para visión artificial, tratamiento de imágenes y Machine Learning. Es especialmente interesante para detectar y leer matrículas mediante modelos de IA.
- **JavaScript / Node.js:** Sería muy útil para crear la parte web o el servidor que recibe los resultados de la cámara, pero no sería nuestra primera opción para desarrollar el reconocimiento de imágenes.
- **R:** Destaca principalmente en estadística y análisis de datos. Para este proyecto tiene menos sentido porque el problema principal es la visión artificial.
- **C++:** Ofrece un rendimiento muy alto y es muy utilizado en sistemas de visión artificial y aplicaciones que necesitan procesar imágenes rápidamente. Lo malo de este, es que es más complejo de desarrollar.
- **PHP:** Puede utilizarse para crear la parte web y gestionar datos, pero tiene menos herramientas específicas para visión artificial e IA.
- **Java:** Es potente y tiene buenas posibilidades para desarrollar aplicaciones y servicios, aunque para trabajar directamente con modelos de visión artificial tiene un ecosistema menos cómodo que Python.

Para decantarnos por un lenguaje en específico, hemos utilizado una matriz de decisión. De esa manera tomamos el lenguaje que encaje mejor con los miembros del equipo y a su vez sea el más óptimo para el desarrollo de la aplicación:

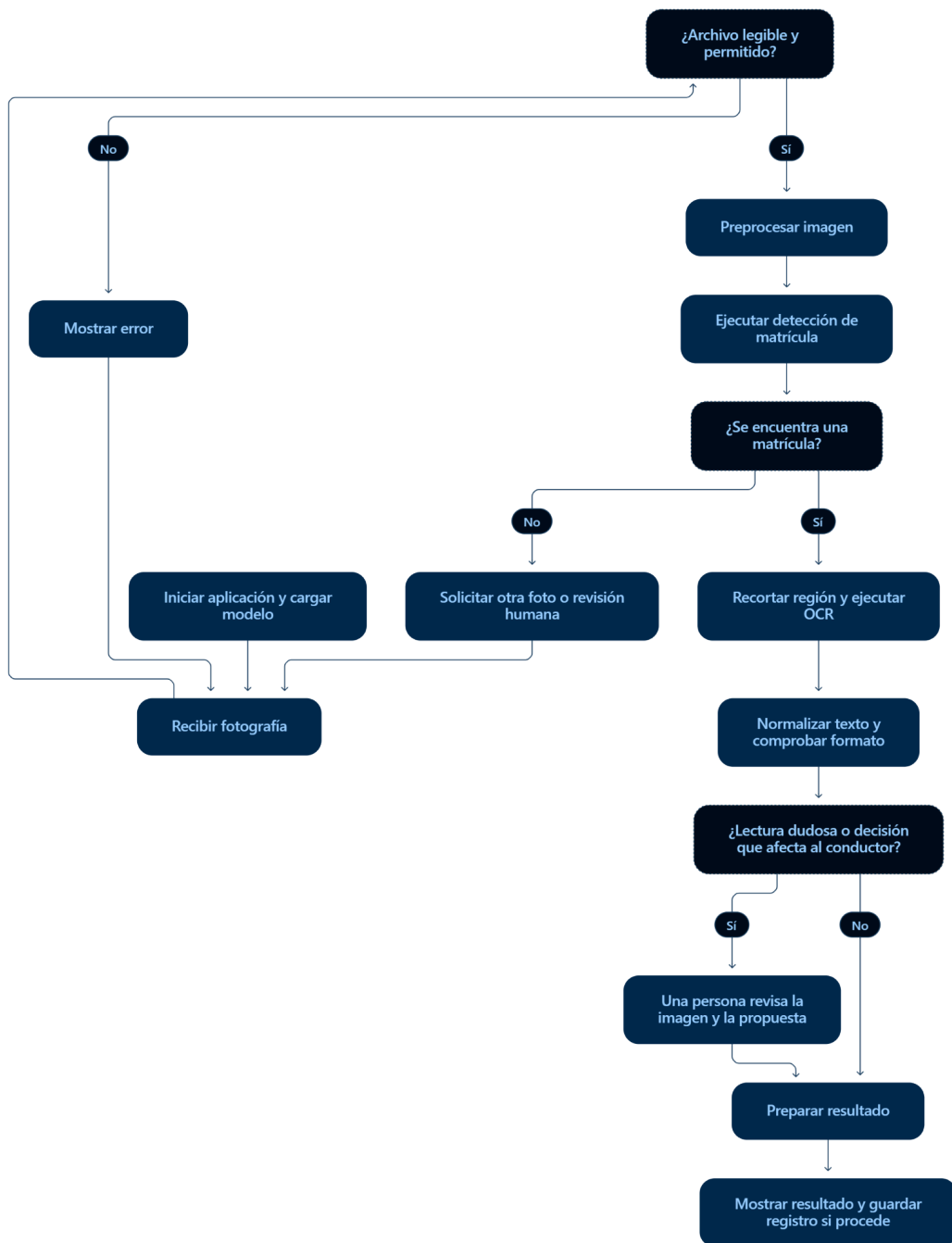
Criterio	Python	JavaScript	R	C ++	PHP	Java
Facilidad de aprendizaje	5	4	4	2	4	3
Legibilidad	5	4	4	3	4	4
Mantenimiento	5	4	3	3	4	5
Integración con web APIs, BD y servicios	5	5	3	3	5	5
Trabajo con datos	5	4	5	3	3	4
Análisis estadístico	5	3	5	2	2	3
Bibliotecas y modelos de IA	5	4	4	4	2	4
Reutilización de modelos preentrenados	5	4	4	4	2	4
Rendimiento, comunidad, despliegue y compatibilidad	5	5	4	5	4	5
Interfaz web	4	5	3	2	5	4
TOTAL	49	42	39	31	35	41

Para la aplicación general elegimos Python como lenguaje principal porque es el que mejor se adapta al tratamiento de imágenes, la conexión con la base de datos y el uso de modelos de inteligencia artificial. Para la parte de IA también elegimos Python, ya que cuenta con bibliotecas y modelos pre entrenados que facilitan la detección y lectura de matrículas.

JavaScript podría utilizarse para crear una interfaz web, y C ++ ofrecería un mayor rendimiento, pero Python resulta más sencillo para nuestro equipo y reúne en un mismo lenguaje las herramientas principales del proyecto. R está más orientado al análisis estadístico, mientras que PHP y Java no ofrecen una ventaja suficiente para este caso. Según nuestra matriz, Python obtiene la puntuación más alta, con 49 puntos.

Programación

Necesidades de la IA en nuestra aplicación:



1. Inicio y captura de la imagen

- **Iniciar aplicación y cargar modelo:** El sistema se inicializa y carga los modelos de Inteligencia Artificial necesarios.
- **Recibir fotografía:** La aplicación recibe el archivo de imagen que contiene la matrícula.

2. Validación inicial del archivo

- **¿Archivo legible y permitido?** El sistema comprueba si el formato de archivo es válido y si la imagen tiene una calidad o legibilidad mínima:
 - **No:** Muestra un error (**Mostrar error**) y retorna al paso de **Recibir fotografía** para intentar cargar un nuevo archivo.
 - **Sí:** Avanza al preprocesamiento.

3. Detección de la matrícula

- En este punto como bien expusimos, tendríamos la primera parte del re entreno, ya que es la visión y detección de la matrícula.
- **Preprocesar imagen:** Se aplican filtros y ajustes (contraste, tamaño, escala de grises, etc.) para optimizar la imagen.
- **Ejecutar detección de matrícula:** Se corre el modelo de detección para ubicar la posición de la placa en la fotografía.
- **¿Se encuentra una matrícula?**
 - **No:** Se solicita una nueva imagen o una revisión manual (**Solicitar otra foto o revisión humana**) y vuelve al punto de **Recibir fotografía**.
 - **Sí:** Continúa hacia el reconocimiento del texto.

4. Extracción y validación del texto (OCR)

- En este punto tendríamos la segunda parte del re entreno, en el que valida la lectura que ha hecho previamente y extrae los datos de la matrícula.
- **Recortar región y ejecutar OCR:** Se recorta únicamente el área donde está la matrícula y se aplica reconocimiento óptico de caracteres (OCR) para extraer el texto.
- **Normalizar texto y comprobar formato:** Se ajustan los caracteres extraídos (espacios, mayúsculas) y se verifica si cumplen con la estructura o formato estándar de una matrícula.
- **Identificación de dónde es la matrícula:** Se identifica a qué localidad pertenece la matrícula.

5. Verificación de confianza y revisión

- **¿Lectura dudosa o decisión que afecta al conductor?** Se evalúa la certidumbre de la lectura o el impacto de la decisión:
 - **Sí:** Requiere intervención manual. Un operador humano valida el resultado (**Una persona revisa la imagen y la propuesta**) antes de avanzar.
 - **No:** El resultado del OCR se da por bueno automáticamente.

6. Finalización y salida

- **Preparar resultado:** Se estructura la información procesada (texto de la matrícula, hora, estado, etc.).
- **Mostrar resultado y guardar registro si procede:** Se despliega la información final en la interfaz y, si aplica, se almacena en la base de datos.

Etapas de antes de utilización de la IA y después

Para la elaboración del programa hemos partido de la división del mismo en dos etapas, antes de la implementación de la ia y posterior a esta misma: Primero un entrenamiento de la ia para que sea capaz de leer la matrícula, en diferentes contextos. Una vez reconocida la imagen, se reconocerá de dónde son esas matrículas, en caso de duda, pasará por una revisión humana.

1. Antes de integrar el modelo de IA

Antes de utilizar el sistema de reconocimiento de matrículas en la aplicación, el equipo tendría que preparar tanto los datos como los diferentes componentes del programa.

Componentes principales

Antes de integrar el modelo, el programa necesitaría preparar varias partes:

Sistema de cámara: encargado de capturar las imágenes de los vehículos.

Control de iluminación: permite detectar cuándo existe poca luz y activar iluminación adicional para mejorar la captura.

Procesamiento de imágenes: prepara las fotografías antes de analizarlas. Por ejemplo, puede modificar el brillo, el contraste o el tamaño de la imagen.

Detector de matrículas: encuentra la posición de la matrícula dentro de la fotografía.

Sistema de reconocimiento de caracteres: transforma los caracteres de la matrícula en texto que pueda utilizar el programa, además identifica de dónde es esa matrícula.

Sistema de validación: Una persona comprueba manualmente si el resultado obtenido tiene sentido y si el modelo tiene suficiente confianza.

Interfaz de usuario: muestra al responsable del aparcamiento la imagen capturada, la matrícula reconocida y el resultado del proceso.

Antes de utilizar el modelo en situaciones reales también sería necesario probarlo con diferentes condiciones: luz, distancia, posición del coche o calidad de imagen. De esta manera se podrían detectar errores antes de poner el sistema en funcionamiento.

2. Después de entrenar o integrar el modelo

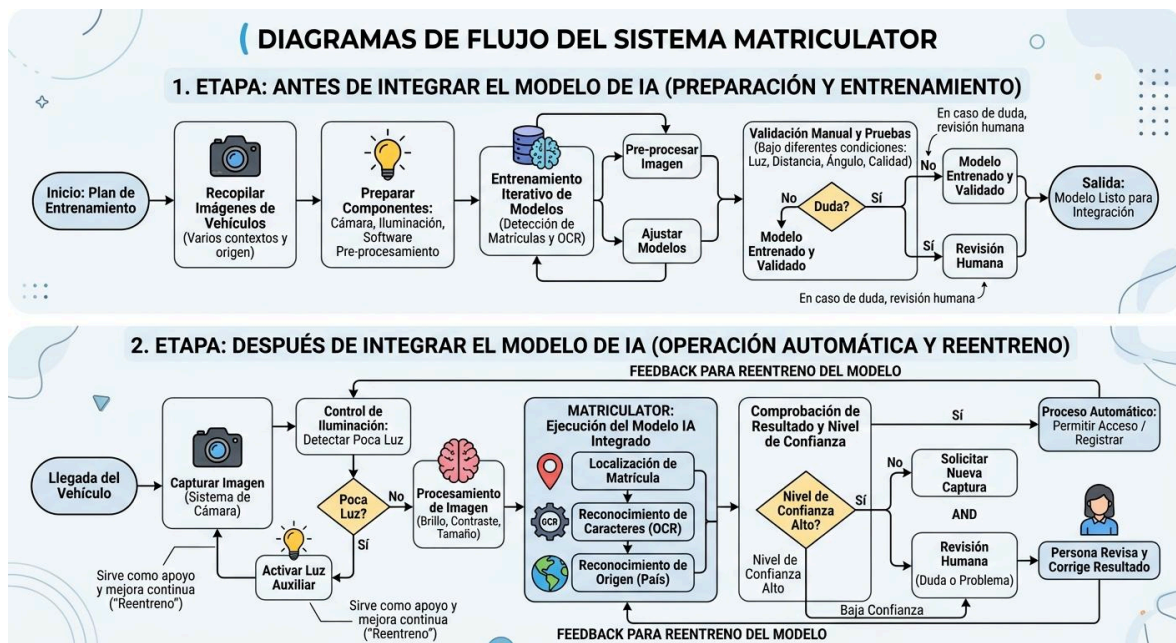
Cuando el modelo ya estuviera disponible dentro de Matriculator, el funcionamiento sería automático en la mayoría de los casos. En este punto el modelo se considera entrenado y funciona de manera autónoma

Diagrama de re entreno

Cuando llega un vehículo, la aplicación recibe la imagen de la cámara y la prepara para el análisis. Después, el modelo localiza la matrícula y el sistema de reconocimiento obtiene los caracteres para así saber a qué localidad pertenece.

El programa comprueba posteriormente si el resultado parece correcto pasa. Cuando el nivel de confianza es suficiente ($>80\%$), el proceso puede continuar automáticamente.

Si la confianza es baja o existe algún problema con la imagen, la aplicación solicita una nueva captura o muestra el caso a una persona para que revise el resultado. De esta manera, la inteligencia artificial sirve como apoyo para automatizar el reconocimiento de matrículas, mientras que la revisión humana permanece disponible para los casos dudosos o cuando una decisión incorrecta pueda afectar al usuario. A esto se le llama **reentreno**.



Pseudocódigo empleado

```
# PSEUDOCÓDIGO - SISTEMA DE DETECCIÓN DE MATRÍCULAS

# 1. Iniciar la aplicación
iniciar_aplicacion()

# 2. Cargar el modelo de Inteligencia Artificial
modelo = cargar_modelo_IA()

# 3. Recibir una fotografía de la cámara
imagen = recibir_fotografia()

# 4. Comprobar que la imagen es válida
def validar_imagen(imagen):
    if imagen no existe:
        return False
    if formato no permitido:
        return False
    if imagen está dañada:
        return False
    return True

# 5. Comprobar la imagen recibida
if validar_imagen(imagen) == False:
    mostrar_error("La imagen no es válida")
    solicitar_nueva_fotografia()
else:
    # 6. Preparar la imagen para la IA
    imagen_preparada = preprocesar_imagen(imagen)
    # 7. Detectar la matrícula mediante IA
    region_matricula = detectar_matricula(
        modelo, imagen_preparada
    )

    # 8. Comprobar si se ha encontrado una matrícula
    if region_matricula no existe:
        solicitar_otra_fotografia()
    else:
        # 9. Recortar la zona de la matrícula
        matricula_recortada = recortar(
            imagen_preparada, region_matricula
        )
        # 10. Leer los caracteres mediante OCR
        texto = ejecutar_OCR(matricula_recortada)
        # 11. Normalizar el texto obtenido
        matricula = normalizar_texto(texto)
```

```

# 12. Comprobar que el formato sea correcto
if formato_matricula_valido(matricula) == False:
    solicitar_revision_humana()
else:
    # 13. Comprobar la confianza de la lectura
    confianza = obtener_confianza()
    # 14. Si la confianza es baja, revisar manualmente
    if confianza < 80:
        resultado = revision_humana(
            imagen, matricula
        )
    else:
        # 15. Comprobar la matrícula
        resultado = comprobar_matricula(matricula)
    # 16. Preparar el resultado final
    preparar_resultado(matricula, resultado)
    # 17. Mostrar el resultado al usuario
    mostrar_resultado(matricula, resultado)
    # 18. Guardar el registro de la operación
    guardar_registro(matricula, resultado)
# 19. Finalizar el proceso
finalizar_proceso()

```

Lenguajes de marcado y formatos de datos

HTML: organiza los elementos de una página web, como los botones, los textos y las imágenes. En Matriculator podría utilizarse para mostrar la interfaz donde el responsable consulta la matrícula y el resultado.

XML: permite guardar e intercambiar información organizada mediante etiquetas. Podría utilizarse si otro sistema del aparcamiento necesitará recibir los datos en este formato.

JSON: permite intercambiar datos de forma estructurada entre partes del programa. Matriculator podría usarlo para enviar el resultado del análisis, como la fecha de captura, la matrícula reconocida, la confianza y el estado.

Markdown: se utiliza para escribir documentación sencilla, como el archivo README del proyecto o las explicaciones de los notebooks.

CSV: organiza datos en filas y columnas. Podría utilizarse para exportar un historial de capturas y resultados y abrirlo después en una hoja de cálculo.

Preguntas a contestar

¿Es una IA débil o fuerte?

Matriculator sería una IA débil, porque estaría diseñada para realizar una tarea concreta: localizar y leer matrículas en imágenes. No tendría inteligencia general ni podría realizar cualquier tipo de tarea.

¿Usamos un modelo pre entrenado o lo entrenamos desde cero?

Al principio se prepararon dos modelos: uno para localizar y reconocer la matrícula, y otro para identificar su tipo y procedencia. Se podrían adaptar modelos pre entrenados con imágenes etiquetadas para cada tarea.

Una vez puesto el sistema en marcha, una persona revisará las lecturas dudosas y corregirá los errores. Esos ejemplos revisados servirían para actualizar los datos de entrenamiento y realizar entrenamientos periódicos. Antes de utilizar una versión nueva, habría que comprobar que funciona correctamente.

Organización

Temporalización del trabajo

Disponíamos de 8 horas para realizar el proyecto, pero finalmente dedicamos unas 12 horas. La siguiente tabla recoge una estimación del tiempo empleado en cada parte:

Tarea	Tiempo previsto	Tiempo real
Leer el enunciado, organizar el trabajo y definir Matriculator	0,5 h	1 h
Comparar lenguajes y elaborar la matriz de decisión	1 h	1,5 h
Recopilar datos de Google Trends y analizar los resultados	0,5 h	0,5 h
Elaborar los diagramas, explicar el funcionamiento y preparar el notebook de pseudocódigo	1 h	1,5 h
Explicar los formatos de datos y preparar el notebook de demostración	1 h	1 h
Crear la página web e incorporar el contenido y los gráficos	1,5 h	2 h
Desarrollar y probar la aplicación funcional	1,5 h	1h
Revisar el informe y corregir diferencias entre los materiales	0,5 h	2,5 h
Total	8 h	12 h

Desviaciones y revisión de la planificación

El principal problema fue que al principio no entendimos bien algunas partes del enunciado. Esto hizo que dedicáramos tiempo a desarrollar una aplicación funcional, aunque la actividad pedía principalmente una propuesta técnica. También tuvimos que rehacer algunas explicaciones y diagramas para ajustarlos a lo solicitado y mantener la coherencia entre el informe y la página web.

Al detectar estas dificultades, tuvimos que ampliar la planificación inicial de 8 a 12 horas, destinando más tiempo al desarrollo, las correcciones y la revisión final.

Reflexión del equipo

Hemos aprendido que antes de empezar a programar debemos revisar los requisitos y distinguir qué entregables son obligatorios y cuáles son mejoras adicionales. Para futuros trabajos, prepararemos una lista de comprobación desde el principio y reservaremos más tiempo para revisar el conjunto. Esto nos ayudará a evitar repetir tareas y a cumplir mejor el plazo previsto.

Reparto de tareas

Integrante	Función principal	Tareas realizadas
Carmelo José Beato Garcia	Organización y gestión	Organizar las tareas y el avance del trabajo, revisar los conceptos y requisitos, coordinar los entregables y elaborar la parte general del informe.
Oihan Cabada	Programación y desarrollo web	Crear la página web, programar la aplicación y redactar las partes del informe relacionadas con el desarrollo y la programación.
Dividimos el trabajo en dos áreas principales: organización y documentación, y desarrollo técnico. Carmelo se centró en organizar lo que íbamos haciendo y preparar los contenidos y entregables. Oihan se centró en crear la página web y programar la aplicación, además de elaborar la documentación correspondiente a su parte.		

Reflexión del equipo

Dividimos el trabajo en dos áreas principales: organización y documentación, y desarrollo técnico. Carmelo se centró en organizar lo que íbamos haciendo y preparar los contenidos y entregables. Oihan se centró en crear la página web y programar la aplicación, además de elaborar la documentación correspondiente a su parte.

EVIDENCIAS IA

Uso de inteligencia artificial generativa

Durante el proyecto utilizamos ChatGPT como apoyo para resolver dudas, organizar contenidos, revisar el informe y trabajar sobre la página web. Carmelo lo utilizó para la organización, los conceptos y los entregables. Oihan también utilizó ChatGPT durante su parte del trabajo, aunque no disponemos de sus conversaciones para incluir sus instrucciones concretas.

Prompts y peticiones de Carmelo: Los siguientes ejemplos proceden de las conversaciones del proyecto. Se incluyen fragmentos de las peticiones originales.

Parte del trabajo	Prompt o fragmento utilizado	Finalidad
Comparación de lenguajes	actualiza la matriz de eleccion con los siguientes criterios	Aportar nuestras valoraciones sobre los seis lenguajes y adaptar la matriz a Matriculator.
Página web	actualiza el site con dicha informacion	Incorporar a la web los contenidos revisados sobre los lenguajes
Control de iluminación	añadir un paso en los cuales si la camara detecta que la luminosidad es insuficiente, enciende una luz	Incluir una condición necesaria para capturar matrículas con poca luz.
Flujo de funcionamiento	Flujo general: Describe el funcionamiento esperado de la aplicación mediante entre 6 y 10 etapas	Pedir ayuda para adaptar los requisitos del enunciado a los diagramas de Matriculator.
Revisión del informe	vamos con la parte del informe, dime en que pagina esta y que cosas pongo	Localizar los apartados que necesitaban ampliaciones y corregirlos por pasos.
Actualización de datos	actualiza el csv de youtube con este que abarca más tiempo	Sustituir los datos por un archivo con un periodo más amplio.
Comprobación de requisitos	que me falta	Comparar nuestro informe con el enunciado y detectar posibles pendientes.

Repreguntas, cambios y correcciones

No utilizamos siempre la primera respuesta de ChatGPT. Fuimos aportando información y solicitando cambios para ajustar las propuestas a nuestro proyecto.

Por ejemplo, aclaramos que nuestra propuesta contemplaba dos entrenamientos iniciales: uno para reconocer matrículas y otro para identificar su procedencia, además de un reentrenamiento posterior con supervisión humana. Con ello corregimos una interpretación que no recogía completamente nuestra idea.

También indicamos que Google Trends ya estaba incluido en la web cuando ChatGPT lo señaló como pendiente al revisar únicamente el PDF. Le proporcionamos el enlace para que tuviera en cuenta ese contenido. Este caso nos mostró que una respuesta puede ser incompleta si la herramienta no dispone de todos los materiales.

Además, pedimos ampliar el periodo del CSV de YouTube, desarrollar mejor las conclusiones y unificar la matriz, porque Python aparecía con 48 puntos en la web y 49 en el informe.

Justificación de la herramienta

Utilizamos ChatGPT porque nos permitía plantear dudas en lenguaje sencillo, aportar documentos y recibir propuestas que podíamos revisar y modificar. Nos ayudó a estructurar explicaciones, comparar nuestro trabajo con los requisitos y preparar cambios concretos.

Las decisiones sobre el proyecto las tomamos nosotros. Aportamos los criterios de comparación, mantuvimos Python como lenguaje principal e indicamos funciones como la iluminación auxiliar y la revisión humana.

Reflexión del equipo

La IA nos ayudó a avanzar en la redacción y a detectar diferencias entre los materiales. Sin embargo, no sustituyó la lectura del enunciado ni evitó que interpretáramos mal algunas partes. Tuvimos que rehacer contenido y dedicar 12 horas en lugar de las 8 previstas.

Aprendimos que debemos comprobar las respuestas con los documentos originales y explicar con precisión lo que queremos. También vimos que una propuesta bien redactada puede contener errores o no coincidir con nuestra idea.